

M06 / PROTECTED ANSWERS

M06 Module Revision Test Answers

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
A01 - DE06-01 Verify WSL2 and choose a safe Linux workspace	3
A02 - DE06-02 Linux navigation and safe file operations	3
A03 - DE06-03 Pipes, redirection, command history and inspection	4
A04 - DE06-04 Permissions, packages, sudo and command safety	4
A05 - DE06-05 Git state: init, status, add, commit, diff and log	5
A06 - DE06-06 Branches, merges and real conflict resolution	5
A07 - DE06-07 GitHub remotes: push, fetch, pull and tracking	6
A08 - DE06-08 Pull requests and review workflow	6
A09 - DE06-09 Python virtual environments inside a repository	7
A10 - DE06-10 .gitignore, secrets and repository hygiene	7
A11 - DE06-11 restore, revert, reflog and safe history repair	8
A12 - DE06-12 Shell scripts, exit codes and environment-driven commands	8
A13 - DE06-13 Repository handoff and clean-clone reproducibility	9

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after a saved closed-book attempt. Each answer gives a concrete solution, a proof path, common mistakes, validation, and a changed retry. Do not overwrite your first attempt.

A01 - DE06-01 | Verify WSL2 and choose a safe Linux workspace

One acceptable solved answer

Verify the shell is actually WSL2/Linux, identify the distro/home, and choose a workspace deliberately. For Linux-heavy repositories, a path under ~/projects avoids many cross-filesystem permission/performance surprises; /mnt/c/... is valid when you intentionally need Windows files.

Worked proof / example

```
uname -a
printf '%s\n' "$HOME"
pwd
mkdir -p ~/projects/de-lab
cd ~/projects/de-lab
```

Common beginner mistakes

Assuming Git Bash or PowerShell is WSL; copying Windows C:\ paths literally into Bash; not knowing whether the repo is on Linux or mounted Windows storage.

Validate your own attempt

In a fresh terminal, prove pwd, distro/kernel and the chosen workspace path; translate one Windows path to /mnt/c form.

Changed retry

Create a second workspace under /mnt/c and explain one reason you might prefer or avoid it.

A02 - DE06-02 | Linux navigation and safe file operations

One acceptable solved answer

Before cp/mv/rm, prove the current directory and target. Use quoted paths and test copy vs move on disposable data. Deletion is not a recycle-bin operation.

Worked proof / example

```
pwd
ls -la
cp 'source file.csv' ready/
ls -l ready/
mv ready/'source file.csv' archive/
ls -l archive/
```

Common beginner mistakes

Running rm with an unexpanded/wrong variable; omitting quotes around spaces; assuming mv keeps a source copy.

Validate your own attempt

Show source/target listings before and after each action and explain which inode/path should exist.

Changed retry

Use one absolute and one relative path in a new source/ready tree; prove the outcome.

A03 - DE06-03 | Pipes, redirection, command history and inspection

One acceptable solved answer

A pipe sends stdout from one process to another. > replaces a file, >> appends, and 2> redirects stderr. Choose based on whether the output is a regeneration or an append-only log.

Worked proof / example

```
grep 'ERROR' app.log | wc -l > error_count.txt
cat missing_file 2> failure.txt
echo 'next run' >> history.log
```

Common beginner mistakes

Using >> for a daily regenerated dataset and duplicating rows; assuming stderr is included in a normal pipe.

Validate your own attempt

Prove a second regeneration with > does not double output and the intentional failure lands in the stderr file.

Changed retry

Filter/count a fresh text file to stdout while redirecting a separate error to stderr.

A04 - DE06-04 | Permissions, packages, sudo and command safety

One acceptable solved answer

Inspect owner/mode/path before changing permissions. For a learner script that lacks owner execute permission, chmod u+x script.sh is narrow and explainable. Do not use chmod 777 or sudo as a generic “make it work” button.

Worked proof / example

```
ls -l script.sh
chmod u-x script.sh
./script.sh # expected permission failure
chmod u+x script.sh
./script.sh
```

Common beginner mistakes

chmod 777; sudo on project files without understanding ownership; changing permissions before confirming the actual error.

Validate your own attempt

Show mode before/after and prove only the intended permission changed.

Changed retry

Repeat on another script and explain why owner execute is sufficient.

A05 - DE06-05 | Git state: init, status, add, commit, diff and log

One acceptable solved answer

Model Git as working tree -> index -> commit. Stage a version, edit again, and prove the later edit is still unstaged after committing the staged snapshot.

Worked proof / example

```
git init
printf 'v1\n' > note.txt
git add note.txt
printf 'v2-uncommitted\n' >> note.txt
git status --short
git diff
git diff --staged
git commit -m 'add v1'
git status --short
```

Common beginner mistakes

Thinking git add permanently tracks all future edits; using only git status without inspecting diffs.

Validate your own attempt

The commit contains the staged state while the later edit remains in the working tree.

Changed retry

Stage only part of a second file set and predict what the commit includes.

A06 - DE06-06 | Branches, merges and real conflict resolution

One acceptable solved answer

Create two branches that change the same logical line, merge, inspect markers, choose the intended project behavior, remove markers and validate the result before committing. A conflict is a content decision, not a syntax-cleanup exercise.

Worked proof / example

```
git switch -c feature
# edit README run command, commit
git switch main
# conflicting edit, commit
git merge feature
# edit conflict intentionally
git add README.md
git commit
```

Common beginner mistakes

Deleting conflict markers without deciding the correct command; accepting “ours/theirs” blindly.

Validate your own attempt

Run or otherwise verify the resolved command and show a clean git status after merge commit.

Changed retry

Resolve a different README/config conflict where the correct answer combines both branches.

A07 - DE06-07 | GitHub remotes: push, fetch, pull and tracking

One acceptable solved answer

origin is a remote name. git fetch updates remote-tracking refs without changing your working tree; git pull fetches then integrates. Use two clones/local bare remote to prove mechanics; label real GitHub authentication separately.

Worked proof / example

```
git clone --bare repo remote.git
git clone remote.git A
git clone remote.git B
# commit+push in A
cd B && git fetch origin
git log --oneline --all --decorate
# worktree unchanged until merge/rebase/pull
```

Common beginner mistakes

Calling a local bare remote “GitHub proof”; assuming fetch changes current files.

Validate your own attempt

Before/after fetch, show B worktree hash/file unchanged while origin/main advances.

Changed retry

Create another commit in A; fetch in B, inspect diff to origin/main, then integrate explicitly.

A08 - DE06-08 | Pull requests and review workflow

One acceptable solved answer

A strong PR proposal contains a focused diff, rationale, validation evidence and response to feedback. If the diff includes a fake .env, block merge, remove/stop tracking it, verify the proposed diff and explain that a real exposed secret would also require rotation.

Worked proof / example

Acceptable review comment: Block: .env contains credential-like configuration. Remove it from the proposed changes, add an ignore rule, and rotate any real secret before merge.

After repair, the PR diff must contain only intended source/config templates and no secret value.

Common beginner mistakes

Merging because tests are green while the diff leaks credentials; calling an offline diff a GitHub PR.

Validate your own attempt

Inspect the exact proposed diff, not just the final working tree; preserve review evidence.

Changed retry

Review a second PR with an accidental generated data file and an untested behavior change.

A09 - DE06-09 | Python virtual environments inside a repository

One acceptable solved answer

Create .venv locally, install from a reproducible requirement/lock source, keep the environment directory untracked, and prove a fresh environment can run the project.

Worked proof / example

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python app.py
git status --short
```

Common beginner mistakes

Committing .venv; relying on global packages; documenting only “pip install whatever is missing”.

Validate your own attempt

Delete/recreate .venv and run from the documented dependency file without copying the old environment.

Changed retry

Clone to a second directory and rebuild the environment from scratch.

A10 - DE06-10 | .gitignore, secrets and repository hygiene

One acceptable solved answer

.gitignore prevents new untracked matches from being added; it does not erase tracked history. For a fake demo secret already tracked, stop tracking the file and prove history still contains the earlier version. For a real secret, rotate/revoke it.

Worked proof / example

```
echo '.env' >> .gitignore
git rm --cached .env
git commit -m 'stop tracking env'
git log --all -- .env
```

Common beginner mistakes

Deleting only the current file and claiming the secret is gone; using a real credential in the exercise.

Validate your own attempt

Current status excludes .env, while history search demonstrates why rotation is required for real exposure.

Changed retry

Repeat with another generated-secret file and document cleanup + rotation decision.

A11 - DE06-11 | restore, revert, reflog and safe history repair

One acceptable solved answer

Use restore for uncommitted working-tree/index correction; revert to add an inverse commit on shared history; reflog to locate recent local reference movement and create a recovery branch. Avoid rewriting shared history reflexively.

Worked proof / example

```
git restore file.txt
git revert <bad-shared-commit>
git reflog --oneline
git branch recovery <reflog-commit>
```

Common beginner mistakes

reset --hard on shared history; confusing restore with revert; assuming reflog is a remote audit log.

Validate your own attempt

Show the shared bad change is reversed by a new commit and the recovery branch points to the intended historical commit.

Changed retry

Create a second bad commit and recover it using a different safe mechanism appropriate to its state.

A12 - DE06-12 | Shell scripts, exit codes and environment-driven commands

One acceptable solved answer

Accept input and OUTPUT_DIR as explicit boundaries, quote variables, write errors to stderr, and return distinct nonzero exit codes for distinct failure classes. Never echo secret values.

Worked proof / example

```
#!/usr/bin/env bash
set -u
input=${1:-}
: "${OUTPUT_DIR:?OUTPUT_DIR required}"
if [[ -z "$input" || ! -f "$input" ]]; then
    echo 'input missing' >&2
    exit 2
fi
mkdir -p "$OUTPUT_DIR"
cp "$input" "$OUTPUT_DIR/"
```

Common beginner mistakes

Unquoted paths; exit 0 after failure; printing environment secrets during debugging.

Validate your own attempt

Test a path with spaces, missing input and missing OUTPUT_DIR; capture stdout/stderr/exit code.

Changed retry

Add a second argument and a distinct validation error with a different exit code.

A13 - DE06-13 | Repository handoff and clean-clone reproducibility

One acceptable solved answer

The clean clone is the handoff proof: clone into a new path, follow README only, recreate environment, supply documented nonsecret config/input, run tests/command and reproduce expected output. No hidden global packages, personal absolute paths or undeclared env vars.

Worked proof / example

Minimum evidence: clean-clone command; dependency/environment recreation; documented input/config; successful test/run output; git status showing no accidental generated/secret files; one deliberate failure/recovery note.

Common beginner mistakes

Testing only in the original working directory; manually copying an undocumented file into the clone; using global packages.

Validate your own attempt

A second person should be able to follow README without asking what file/path/package is missing.

Changed retry

Create a fresh shipment repository with a changed wrapper/config and repeat the clean-clone proof.